MATHEMATICS / SOFTWARE DESIGN / REVERSE ENGINEERING

F-1: An Educational Reversible Mathematical Cipher

From an inverse function to a tested software model

AUTHOR Ayoub El Maadi

PROMPT / PROJECT DESCRIPTION An independent paper explaining the mathematical model, implementation decisions, and verification strategy of an educational reversible cipher developed during my study of reverse engineering.

Abstract

F-1 is a small educational program that turns a mathematical idea - the relationship between a function and its inverse - into a reversible character transformation. It uses the positive divisors of 27,720 to create a finite codebook, applies a rational function whose outputs are exact integers, and then maps those values to printable characters. The project was developed as a practical exercise during my study of reverse engineering: rather than treating a transformation as a one-way operation, I wanted to understand what information makes it possible to reconstruct the original input. This paper explains the mathematics, the software structure, the verification strategy, and the limits of the design. F-1 is a learning tool, not secure cryptography.

1. The question behind the project

While studying reverse engineering, I became interested in a basic but important question: when a program transforms information, how can we reason about the transformation in the opposite direction? Reverse engineering often involves examining how a system behaves, identifying the steps that produce an output, and testing whether an explanation of that behaviour is correct. I used this way of thinking as the starting point for a small mathematical experiment.

The experiment became F-1. Its goal was deliberately limited. I did not set out to build a new cryptographic standard or to protect real messages. I wanted to construct a reversible mapping, derive its inverse, implement both directions using exact arithmetic, and test that the original input could always be recovered for every supported character.

2. Choosing a function with an exact inverse

The mathematical engine is based on the constant K = 27,720 and the function:

$$f(x) = (x - K) / x = 1 - K/x$$

The corresponding inverse is:

$$f^{-1}(y) = K / (1 - y)$$

The inverse follows directly from the original equation. Starting with y = (x - K)/x, I rewrite it as y = 1 - K/x. Subtracting y from 1 gives 1 - y = K/x. Multiplying by x and dividing by 1 - y gives x = K/(1 - y). The result is the inverse function, which reconstructs x from a valid output y.

The important implementation decision was to avoid approximate arithmetic. A general input x could make K/x a fraction, but F-1 does not use arbitrary values. It selects x only from the positive divisors of K, excluding 1. Because each selected x divides K exactly, K/x is an integer. Therefore f(x) is also an integer. The inverse returns the original divisor exactly when it is given an output produced by the forward function.

The number 27,720 has the prime factorization $2^3 \times 3^2 \times 5 \times 7 \times 11$. Its number of positive divisors is therefore (3 + 1)(2 + 1)(1 + 1)(1 + 1)(1 + 1) = 96. Excluding the divisor 1 leaves 95 usable values. This count was not incidental: it matches the 95 printable ASCII characters from code 32 (space) through code 126 (~).

3. Turning the function into a character mapping

F-1 uses the 95 selected divisors as a finite mathematical codebook. Each plaintext character is assigned one divisor x. The program applies f(x) to obtain an integer y, and a second one-to-one mapping associates that y value with a printable ciphertext character. Decoding follows the path in reverse: the ciphertext character identifies y, the inverse function recovers x, and the codebook identifies the original character.

For example, 360 is a divisor of 27,720. Since 27,720 / 360 = 77, the forward transformation gives:

$$f(360) = 1 - 77 = -76$$

Applying the inverse recovers the starting value:

$$f^{-1}(-76) = 27{,}720 / (1 - (-76)) = 27{,}720 / 77 = 360$$

This example shows the central property of the project: the reverse operation is not an estimate. It returns the exact integer used in the forward operation. The same principle applies across the full codebook, not just to this one example.

4. The role of the seed and the software structure

F-1 has a default seed of 20261001. The seed changes the assignment of characters to codebook values and the assignment of transformed values to ciphertext characters. It does not change K, f(x), or $f^{-1}(y)$. Keeping this distinction explicit was important: the seed controls the arrangement of the codebook, while the mathematical transformation remains fixed. Using the same seed reproduces the same mapping; choosing a different seed changes the mapping.

I kept the mathematical engine in logic.py as the single source of truth. It defines K, the supported alphabet, the forward and inverse functions, divisor generation, the F1Cipher class, and methods for encoding, decoding, explaining a character transformation, and displaying the mapping table. This separation makes it possible to inspect and test the mathematical core without depending on a graphical interface.

The project also separates the core from its presentation layer. The graphical laboratory described in the project documentation does not implement a second version of the mathematics. Instead, it passes requests through a CipherService bridge to the same engine. This reduces the risk that the interface and the command-line program will produce different results because they use separate implementations.

5. Verification: checking more than a successful example

A reversible design should not be judged by whether one short message can be encoded and decoded. I therefore organized the verification around mathematical invariants and expected program behaviour. The project documentation describes 28 test categories in the core self-test and a pytest suite containing 40 test cases.

The first group checks the mathematical and codebook assumptions: the value of K, the divisor count, the number and uniqueness of selected divisors, integer-valued outputs, uniqueness of transformed values, and the inverse relationship across the full codebook. Another group checks reversibility for every supported printable character, fixed messages, randomized messages, and long messages. Further checks cover seed reproducibility, the fact that changing the seed does not change the mathematical functions, rejection of unsupported Unicode characters, malformed ciphertext, and agreement between the character explanation and the actual encoder and decoder.

The project also checks round-trip properties: decoding an encoded character must return that character, and decoding an encoded message must return the original message. These properties express the main claim of F-1 in a form that can be tested repeatedly. The implementation uses integer operations only in the mathematical core; it does not rely on floating-point values, rounding, or truncation.

6. What F-1 does not claim

The name “cipher” describes the educational transformation, not a claim of practical security. F-1 is not a modern cryptographic algorithm and must not be used to protect real data. Its character mapping is deterministic for a given seed: the same plaintext character receives the same ciphertext character under that mapping. That structure can reveal patterns in messages, and the seed is not a substitute for a reviewed cryptographic key-management design. The project does not provide the security properties expected from established encryption systems.

Stating this limitation is part of the project rather than a footnote to it. Mathematical reversibility and cryptographic security are different properties. A function can have a clean inverse and still be unsuitable for protecting information against an intelligent attacker. F-1 demonstrates the first property; it does not establish the second.

7. From the mathematical core to a usable tool

The public interface is designed to make the same mathematical engine usable in more than one way. At the character level, encode_char and decode_char expose the forward and reverse transformations. At the message level, encode and decode apply those operations to a complete string. The explain_char method provides a trace of one character through the model, while mapping_table makes the full codebook inspectable. These methods are useful for learning because they expose the intermediate values instead of presenting encoding as a mysterious one-step operation.

The command-line interface supports encoding and decoding text, selecting a seed, and reading input from standard input. The documented examples include encoding HELLO with the default seed and then decoding the resulting ciphertext to recover HELLO. Running the core module without arguments invokes its verification routine, while the pytest wrapper provides an additional way to run the test cases. This makes the project usable both as a small program and as a laboratory for examining its own behaviour.

The input rules are intentionally explicit. The supported alphabet is printable ASCII, from space through the tilde character. Unsupported characters are rejected rather than silently altered. Non-string inputs raise a type error, and malformed ciphertext is rejected. The graphical laboratory described in the project documentation adds a more visual way to inspect the same sequence, including the forward and reverse pipeline, character details, the searchable codebook, and a verification panel. It remains a presentation layer: the mathematical source of truth stays in logic.py.

8. What I learned

Building F-1 helped me connect several kinds of reasoning. The mathematical work required deriving an inverse and selecting a domain in which every operation remains exact. The programming work required turning that model into a consistent codebook and separating the engine from the interface. The testing work required expressing the design as properties that should hold for every supported input, rather than relying on a few demonstrations.

The connection to reverse engineering is in the way I approached the problem: I treated a transformation as something that should be understood in both directions, then used explicit invariants to check whether the explanation was correct. F-1 is not a tool for reverse engineering binaries or breaking existing encryption. It is a small, controlled project through which I explored how mathematical structure, program behaviour, and verification fit together.

The most useful lesson was that a convincing demonstration is not the same as a reliable system. A successful example can show that an idea works once; a defined domain, a derived inverse, and systematic tests provide stronger reasons to trust that the implementation behaves as intended. That distinction is one I want to carry into future work in computer science and cybersecurity.